

# Arquitectura de Computadores

## Resumen de la clase del 30 de abril de 2026

*Tema general: redondeo IEEE 754, coprocesadores/FPU, taxonomía de Flynn y AVX/SIMD*

Este resumen está orientado a repasar rápido para trabajo práctico o consultas. No es una transcripción literal: organiza la clase por temas y resalta lo que probablemente importa para resolver ejercicios o responder preguntas del profesor.

### 1. Idea central de la clase

- La clase empieza cerrando el tema de redondeo de números de punto flotante, especialmente cómo se comportan los modos de redondeo cuando el número es positivo o negativo.
- Luego cambia hacia la historia del procesamiento de punto flotante en Intel: primero como coprocesador separado, después integrado en el procesador principal.
- Finalmente conecta el punto flotante moderno con ejecución paralela/vectorial: taxonomía de Flynn, SIMD y tecnologías AVX.

### 2. Redondeo en punto flotante

Se repasaron cuatro modos de redondeo. La confusión típica aparece con números negativos: “hacia  $+\infty$ ” no significa “hacer el valor absoluto más grande”, sino escoger el representable más cercano por encima en la recta numérica.

- **Hacia el más próximo:** elige el número representable más cercano.
- **Hacia cero:** truncamiento; elimina los dígitos sobrantes sin alejarse de 0.
- **Hacia  $+\infty$ :** redondeo por exceso; busca el siguiente representable mayor o igual.
- **Hacia  $-\infty$ :** redondeo por defecto; busca el siguiente representable menor o igual.

Ejemplo usado en clase: representación con 4 dígitos en la parte fraccionaria y 3 dígitos de guarda.

| Número     | Más próximo | Hacia cero | Hacia $+\infty$ | Hacia $-\infty$ |
|------------|-------------|------------|-----------------|-----------------|
| 1.1234765  | 1.1235      | 1.1234     | 1.1235          | 1.1234          |
| -1.1234765 | -1.1235     | -1.1234    | -1.1234         | -1.1235         |
| 5.4326398  | 5.4326      | 5.4326     | 5.4327          | 5.4326          |
| -5.4326398 | -5.4326     | -5.4326    | -5.4326         | -5.4327         |

Regla rápida: para positivos, hacia  $+\infty$  se parece a “subir” y hacia  $-\infty$  se parece a “bajar”. Para negativos, hacia  $+\infty$  acerca el número a cero, mientras que hacia  $-\infty$  lo hace más negativo.

### 3. Programación en punto flotante original

- La clase presenta la etapa inicial del punto flotante en Intel con coprocesadores matemáticos separados del CPU principal.
- El ejemplo mencionado fue el coprocesador 8087. La idea importante no es memorizar el chip por sí solo, sino entender que el punto flotante era una unidad especializada aparte.

- Durante los 80 se continuó esa línea con coprocesadores como 80287 y 80387, acompañando la evolución de la familia 80x86.
- Mantener separada la unidad de punto flotante permitía vender procesadores principales más baratos y dejar el coprocesador como opción para equipos que necesitaran mayor capacidad numérica.

## 4. Evolución de mercado e integración de la FPU

- Con el aumento de capacidades de las PC, las empresas y usuarios empezaron a valorar más el rendimiento y menos solamente el precio.
- Equipos con procesadores como el 80386 empezaron a reemplazar computadores más costosos de fabricantes tradicionales en ciertos entornos.
- Programas como hojas de cálculo y procesadores de texto impulsaron la adopción de computadoras personales en empresas y hogares.
- La consecuencia clave fue que Intel pudo apostar por procesadores más poderosos, integrando finalmente la FPU dentro del procesador principal.

## 5. Taxonomía de Flynn

La taxonomía de Flynn clasifica arquitecturas según cuántos flujos de instrucciones y cuántos flujos de datos manejan. Es importante porque prepara el camino para entender SIMD y AVX.

| Categoría | Significado                         | Idea práctica                                                                                         |
|-----------|-------------------------------------|-------------------------------------------------------------------------------------------------------|
| SISD      | Single Instruction, Single Data     | Modelo secuencial clásico: una instrucción opera sobre un dato o flujo de datos a la vez.             |
| SIMD      | Single Instruction, Multiple Data   | Una misma instrucción se aplica simultáneamente sobre varios datos. Base de la computación vectorial. |
| MISD      | Multiple Instruction, Single Data   | Varias instrucciones sobre un mismo flujo de datos. Es poco común en arquitecturas generales.         |
| MIMD      | Multiple Instruction, Multiple Data | Varios procesadores/núcleos ejecutan instrucciones distintas sobre datos distintos.                   |

Para esta clase, la categoría más importante es SIMD, porque explica cómo un procesador puede acelerar operaciones repetidas sobre vectores, matrices o datos empaquetados.

## 6. SIMD y computación vectorial

- La solución moderna para acelerar punto flotante fue aplicar SIMD a las unidades de punto flotante.
- En este contexto se habla de computación vectorial: una sola instrucción puede procesar varios elementos de un vector a la vez.
- Esto beneficia especialmente operaciones de álgebra lineal, gráficos, multimedia, ciencia de datos, simulación y cualquier tarea con muchos datos numéricos repetitivos.

## 7. AVX y datos empaquetados

- AVX utiliza registros de 256 bits.
- La característica central es que un registro puede contener más de un dato al mismo tiempo; a esto se le llama datos empaquetados.
- Cada registro AVX puede contener enteros o datos de punto flotante, pero no ambos simultáneamente como una misma operación homogénea.

| Tipo de dato             | Cantidad que cabe en 256 bits | Razón                                          |
|--------------------------|-------------------------------|------------------------------------------------|
| double / quadword        | 4                             | $4 \times 64 \text{ bits} = 256 \text{ bits}$  |
| float / int / doubleword | 8                             | $8 \times 32 \text{ bits} = 256 \text{ bits}$  |
| float16 / short / word   | 16                            | $16 \times 16 \text{ bits} = 256 \text{ bits}$ |
| char / byte              | 32                            | $32 \times 8 \text{ bits} = 256 \text{ bits}$  |

## 8. AVX, AVX2 y AVX-512

La clase muestra tablas comparativas. La idea importante es ver la evolución: AVX introduce el modelo vectorial moderno; AVX2 amplía soporte, especialmente para enteros empaquetados de 256 bits; AVX-512 expande a registros/operaciones de 512 bits y agrega más control.

- **AVX:** soporta sintaxis de tres operandos, operaciones SIMD con enteros empaquetados de 128 bits y operaciones de punto flotante empaquetadas de 128/256 bits.
- **AVX2:** agrega más operaciones SIMD, especialmente enteros empaquetados de 256 bits, broadcast, gather y fused multiply-add.
- **AVX-512:** agrega operaciones de 512 bits, más conversiones/permutaciones, control de redondeo a nivel de instrucción, scatter y ejecución condicional con registros opmask.

Vocabulario de la tabla: SPFP significa Single Precision Floating Point; DPFP significa Double Precision Floating Point. FMA o fused multiply-add combina multiplicación y suma en una operación del tipo  $a*b + c$ , útil en álgebra lineal y cómputo numérico.

## 9. Conexión con las clases anteriores

- En las clases anteriores se estudió cómo se representan los flotantes con IEEE 754: signo, exponente, sesgo, mantisa, números especiales, error y underflow.
- Esta clase explica por qué el hardware necesita mecanismos especializados para trabajar con esos números de forma eficiente.
- El camino conceptual es: representación IEEE 754 -> redondeo/error -> FPU/coprocesador -> integración en CPU -> SIMD/AVX para procesar muchos flotantes a la vez.

## 10. Cosas que conviene memorizar para el trabajo

- Hacia cero = truncamiento.
- Hacia  $+\infty$  y hacia  $-\infty$  dependen de la recta numérica; con negativos, no funcionan igual que con positivos.
- SIMD = una instrucción aplicada a múltiples datos.

- AVX usa registros de 256 bits; AVX-512 llega a 512 bits.
- En 256 bits caben 4 doubles, 8 floats/ints, 16 shorts o 32 bytes.
- La FPU primero fue un coprocesador separado y luego se integró al procesador principal.
- La computación vectorial acelera operaciones repetidas sobre arreglos, vectores y matrices.

## 11. Preguntas rápidas de práctica

- Si se trabaja con 4 decimales, ¿cómo se redondea -1.1234765 hacia  $+\infty$ ? Respuesta: -1.1234, porque es el representable mayor.
- ¿Qué diferencia principal hay entre SISD y SIMD? SISD ejecuta una instrucción sobre un flujo de datos; SIMD ejecuta una misma instrucción sobre múltiples datos.
- ¿Cuántos float de 32 bits caben en un registro AVX de 256 bits? 8.
- ¿Por qué la FPU se integró al procesador? Porque el mercado empezó a valorar más la capacidad/rendimiento y el punto flotante se volvió importante para software real.

## 12. Resumen ultra corto

La clase conecta dos mundos: primero, cómo se redondean y manejan los números de punto flotante; segundo, cómo el hardware moderno acelera esos cálculos. Históricamente, Intel usó coprocesadores de punto flotante separados, luego integró la FPU al CPU y finalmente avanzó hacia procesamiento vectorial con SIMD/AVX. Para el trabajo, lo esencial es dominar los modos de redondeo, entender la taxonomía de Flynn y saber que AVX permite operar con varios datos empaquetados dentro de un mismo registro.